LAPORAN PRAKTIKUM ANALISIS KOMPLEKSITAS ALGORITMA

Disusun Oleh:

Nama	: Aulia illah
Nim	: 2025573010070
Kelas	: TI-1C
Dosen	: Muhammad Reza Zulman, S.ST., M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

PRODI TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

2025-2026

Latar Belakang

Dalam perkembangan teknologi informasi yang semakin pesat, kebutuhan akan aplikasi yang cepat dan efisien menjadi sangat penting. Setiap program yang dibuat tidak hanya dituntut untuk berjalan dengan benar, tetapi juga harus mampu menangani data dalam jumlah besar dengan waktu yang optimal dan penggunaan memori yang efisien.

Seringkali, sebuah algoritma yang terlihat sederhana dapat menjadi sangat lambat ketika digunakan pada data berskala besar. Hal ini disebabkan oleh kompleksitas algoritma yang tidak efisien. Oleh karena itu, diperlukan suatu metode untuk menganalisis dan mengukur performa algoritma, yaitu melalui konsep Big O Notation.

Big O Notation digunakan untuk menggambarkan seberapa besar pertumbuhan waktu eksekusi atau penggunaan memori suatu algoritma terhadap ukuran input. Dengan memahami konsep ini, seorang programmer dapat memilih algoritma yang lebih baik serta menghindari penggunaan algoritma yang memiliki kompleksitas tinggi seperti $O(n^2)$ atau $O(n^3)$.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami pentingnya analisis kompleksitas algoritma serta mampu menerapkannya dalam pemrograman menggunakan JavaScript dan Node.js.

1. Tujuan Praktikum
   - Memahami mengapa analisis efisiensi algoritma itu penting
   - Menjelaskan konsep Time Complexity dan Space Complexity
   - Membaca dan menulis notasi Big O: O(1), O(log n), O(n), O(n log n), O(n2)
   - Menganalisis kompleksitas loop tunggal dan nested loop
   - Membandingkan efisiensi dua algoritma secara empiris menggunakan Date.now()
   - Mengenali pola kode yang memiliki kompleksitas buruk dan cara memperbaikinya

2. Langkah Kerja:

Buat folder praktikum-4/ dan masuk ke dalamnya dan Inisialisasi npm project.

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat (main)
$ mkdir praktikum-5

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat (main)
$ cd praktikum-5

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ npm init -y
Wrote to D:\2025573010070_praktikum_algoritma_dan_struktur_dat\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
```

A. Mengapa Efisiensi Algoritma Penting?

Skenario: Mencari Nama di Daftar

Bayangkan kamu punya daftar 10 nama. Mencarinya satu per satu paling lama butuh 10 langkah — tidak masalah. Sekarang bayangkan aplikasi seperti Google dengan miliaran data. Algoritma yang 'cukup baik' untuk 10 data bisa membutuhkan jutaan tahun untuk miliaran data jika algoritmanya buruk. Itulah mengapa kita perlu cara formal untuk mengukur efisiensi: Big O Notation. Big O menjawab pertanyaan: 'Bagaimana waktu/memori yang dibutuhkan algoritma bertumbuh seiring bertambahnya data input?'

Perlu diingat: Big O mengukur laju pertumbuhan (growth rate), bukan waktu absolut dalam detik,

karena waktu absolut bergantung pada hardware.

1. Langkah 1 Pastikan kamu berada di dalam folder praktikum-5/. Buat file baru bernama 01-introkompleksitas.js.

```
touch 01-intro-kompleksitas.js
```

2. Langkah 2 Buka file 01-intro-kompleksitas.js di VS Code, lalu ketikkan kode berikut.

```
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris
// Fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}
const N = 100_000; // ukuran data: 100 ribu
// --- Algoritma A: O(n) — loop satu kali ---
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}
// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n) {
```

```
  return (n * (n + 1)) / 2;
}
// --- Algoritma C: O(n2) — loop bersarang ---
function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}
const data = Array.from({ length: 5000 }, (_, i) => i);
console.log("=== Perbandingan Waktu Eksekusi ===");
ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));
console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

3. Simpan file, lalu jalankan dari terminal.

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 1 ms
O(n2) cariPasangan : 17 ms

Hasil sama? true
```

B. Notasi Big O

Hirarki Kompleksitas Big O

Berikut kelas-kelas Big O dari yang paling cepat ke paling lambat:

- O(1) — Konstan. Waktu eksekusi tidak bergantung pada ukuran input sama sekali.
- Contoh: akses elemen array dengan indeks, operasi aritmatika, akses property object.
- O(log n) — Logaritmik. Setiap langkah memotong masalah menjadi separuh.
- Contoh: Binary Search — mencari di 1 juta data hanya butuh ~20 langkah.
- O(n) — Linear. Waktu tumbuh sebanding lurus dengan ukuran input.

▪ Contoh: loop sekali melewati array, Linear Search.

▪ O(n log n) — Linearitmik. Sedikit lebih buruk dari linear.

▪ Contoh: Merge Sort, Quick Sort — algoritma sorting terbaik yang bisa dicapai.

▪ O(n2) — Kuadratik. Nested loop — sangat lambat untuk data besar.

▪ Contoh: Bubble Sort, Selection Sort, nested loop dua tingkat.

▪ O(2n) — Eksponensial. Sangat buruk — hanya bisa digunakan untuk n sangat kecil.

▪ Contoh: Fibonacci rekursif naif, subset generation.

Aturan penyederhanaan Big O:

1. Buang konstanta: O(2n) → O(n), O(100) → O(1)

2. Buang suku kecil: O(n2 + n) → O(n2), O(n + log n) → O(n)

3. Perhatikan suku yang paling dominan seiring n → ∞

1. Buat file baru bernama 02-kelas-big-o.js di dalam folder praktikum-5/.

```
touch 02-destructuring-spread.js
```

2. Buka file 02-kelas-big-o.js di VS Code, lalu ketikkan kode berikut

```
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O
// ─── O(1) — Konstan ──────────────────────────────
function ambilPertama(arr) {
  return arr[0];
} // O(1)
function tambahkanItem(arr, item) {
  arr.push(item);
} // O(1)
function isGenap(n) {
  return n % 2 === 0;
} // O(1)
// ─── O(log n) — Logaritmik ───────────────────────
function binarySearch(arr, target) {
  let kiri = 0,
    kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
```

```
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}
// ─── O(n) — Linear ──────────────────────────────────
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}
// ─── O(n2) — Kuadratik ──────────────────────────────
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
  return a;
}
// ─── O(2n) — Eksponensial ───────────────────────────
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
}
// ── Demonstrasi ──────────────────────────────────────
console.log("=== O(1) — selalu cepat ===");
console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
console.log(isGenap(42)); // true
console.log("\n=== O(log n) — Binary Search ===");
const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
binarySearch(sorted, 731_452); // cari di 1 juta data
console.log("\n=== O(n) — Linear Search ===");
console.log(
  "Max dari 1000 elemen:",
  cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
);
console.log("\n=== O(n2) — Bubble Sort (kecil saja) ===");
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
console.log("\n=== O(2n) — Fibonacci Rekursif (n kecil!) ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
```

```
console.log("");
// Demonstrasi betapa lambatnya O(2n)
console.log("\nWaktu fib(35) O(2n):");
let t = Date.now();
fibRekursif(35);
console.log(Date.now() - t, "ms");
console.log("Waktu fib(35) O(n) memoization:");
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
  return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
}
t = Date.now();
fibMemo(35);
console.log(Date.now() - t, "ms");
```

3. Langkah 3 Simpan file, lalu jalankan dari terminal

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
 Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n2) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2n) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2n):
86 ms
Waktu fib(35) O(n) memoization:
0 ms
```

Latihan 1: Identifikasi Kompleksitas
1. Buat file baru latihan-1.js.

2. Untuk setiap potongan kode di bawah ini, tulis dalam komentar Big O-nya dan alasannya:

3. Fungsi A: function fn(n) { return n * 2; } → Big O: ?

4. Fungsi B: for (let i=0; i<n; i++) for (let j=0; j<n; j++) console.log(i,j); → Big O: ?

5. Fungsi C: for (let i=1; i<n; i*=2) console.log(i); → Big O: ?

6. Fungsi D: arr.forEach(x => arr.forEach(y => arr.forEach(z => console.log(x,y,z)))); → Big O: ?

7. Implementasikan dan jalankan setiap fungsi, ukur waktunya dengan Date.now() untuk

n=1000.

8. Buat function hitungKompleksitas(n, fn) yang menjalankan fn(n) dan mencetak waktu eksekusi.

4. Buat file baru latihan-1.js.

```
Touch latihan-1.js
```

5. Buka file Latihan-1.js di VS Code, lalu ketikkan kode berikut

```
// Latihan 1: Identifikasi Kompleksitas

// Fungsi A
// Big O: O(1)
// Alasan: Operasi konstan, tidak bergantung pada n
function fnA(n) {
  return n * 2;
}

// Fungsi B
// Big O: O(n^2)
// Alasan: Dua perulangan bersarang (nested loop)
function fnB(n) {
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      // console.log(i, j);
    }
  }
}

// Fungsi C
// Big O: O(log n)
// Alasan: i dikali 2 setiap iterasi (pertumbuhan eksponensial)
```

```javascript
function fnC(n) {
  for (let i = 1; i < n; i *= 2) {
    // console.log(i);
  }
}

// Fungsi D
// Big O: O(n^3)
// Alasan: Tiga perulangan bersarang
function fnD(n) {
  let arr = Array.from({ length: n }, (_, i) => i);

  arr.forEach((x) => {
    arr.forEach((y) => {
      arr.forEach((z) => {
        // console.log(x, y, z);
      });
    });
  });
}

// Fungsi untuk menghitung waktu eksekusi
function hitungKompleksitas(n, fn) {
  const start = Date.now();
  fn(n);
  const end = Date.now();
  console.log(fn.name + " => " + (end - start) + " ms");
}

// Pengujian
const n = 1000;

hitungKompleksitas(n, fnA);
hitungKompleksitas(n, fnB);
hitungKompleksitas(n, fnC);

// Hati-hati: fnD sangat berat jika n besar
hitungKompleksitas(50, fnD);
```

6. Jalankan di terminal

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ node Latihan1.js
fnA => 0 ms
fnB => 3 ms
fnC => 0 ms
fnD => 1 ms
```

C. Space Complexity

Space Complexity

Selain waktu, kita juga harus mempertimbangkan memori yang digunakan algoritma. Inilah yang disebut Space Complexity.

Space complexity mengukur memori tambahan (auxiliary space) yang dibutuhkan algoritma seiring bertambahnya input — tidak termasuk input itu sendiri.

Contoh perbandingan:

- Algoritma yang hanya menggunakan beberapa variabel → O(1) space
- Algoritma yang membuat array baru sebesar n → O(n) space
- Rekursi sedalam n tingkat → O(n) space (call stack)

Trade-off yang umum: Banyak algoritma bisa dibuat lebih cepat dengan mengorbankan lebih banyak memori, atau sebaliknya. Ini disebut time-space trade-off.

Contoh: memoization membuat Fibonacci dari O(2n) time → O(n) time, tapi memerlukan O(n) space
tambahan.

1. Buat file baru bernama 03-space-complexity.js di dalam folder praktikum-5/.

```
touch 03-space-complexity.js
```

2. Langkah 2 Buka file 03-space-complexity.js di VS Code, lalu ketikkan kode berikut

```
// 04-space-complexity.js
// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
  let total = 0; // hanya 1 variabel tambahan
  for (const x of arr) total += x;
  return total;
```

```
}
// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
  const baru = []; // array baru tumbuh seiring arr
  for (const x of arr) baru.push(x * 2);
  return baru;
}
// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1); // n frame di call stack
}
// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
  return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
  const seen = new Set(); // Set tumbuh hingga n elemen
  for (const x of arr) seen.add(x);
  return seen.size;
}
const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
console.log("Elemen unik:", hitungUnik(dataAcak)); // 7
```

3. Simpan file, lalu jalankan dari terminal

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
```

4. Latihan 3: Class BankAccount

1. Tambahkan kode di bawah file 04-space-complexity.js.

2. Buat dua fungsi untuk menemukan dua angka dalam array yang jumlahnya = target:

▪ cariPasanganLambat(arr, target) — gunakan nested loop. Tulis Big O time & space-nya dalam komentar.

▪ cariPasanganCepat(arr, target) — gunakan Set untuk menyimpan angka yang sudah dilihat. Cek apakah target - elemen ada di Set. Tulis Big O-nya.

3. Uji kedua fungsi dengan [2, 7, 11, 15] dan target 9. Keduanya harus mengembalikan [2, 7].

4. Ukur waktu keduanya dengan array berisi 50.000 angka acak dan target yang tidak ada di array.

5. Diskusi: Fungsi mana yang lebih baik? Apa trade-off-nya?

```
console.log("\n \n latihan");
// Latihan 2: Space & Time Complexity

// Fungsi 1: Cara Lambat (Nested Loop)
// Big O Time: O(n^2)
// Big O Space: O(1)
// Alasan: dua loop bersarang, tanpa struktur tambahan
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

// Fungsi 2: Cara Cepat (Menggunakan Set)
// Big O Time: O(n)
// Big O Space: O(n)
// Alasan: satu loop, tapi pakai Set untuk menyimpan data
function cariPasanganCepat(arr, target) {
  const seen = new Set();

  for (let i = 0; i < arr.length; i++) {
```

```
    const selisih = target - arr[i];

    if (seen.has(selisih)) {
      return [selisih, arr[i]];
    }

    seen.add(arr[i]);
  }

  return null;
}

// Pengujian kecil (sesuai soal)
const smallArr = [2, 7, 11, 15];
const target = 9;

console.log("Hasil Lambat:", cariPasanganLambat(arr, target)); // [2,7]
console.log("Hasil Cepat :", cariPasanganCepat(arr, target)); // [2,7]

// Pengujian data besar (50.000)

// generate data acak
const arrBesar = Array.from({ length: 50000 }, () =>
  Math.floor(Math.random() * 100000),
);

const targetBesar = -1; // kemungkinan tidak ada

// fungsi ukur waktu
function ukurWaktu(fn, arr, target) {
  const start = Date.now();
  const hasil = fn(arr, target);
  const end = Date.now();
  console.log(fn.name + " => " + (end - start) + " ms | hasil:", hasil);
}

// Jalankan pengujian

console.log("\n=== Pengujian Data Besar ===");

// Fungsi lambat pakai data kecil agar tidak hang
ukurWaktu(cariPasanganLambat, arrBesar.slice(0, 2000), targetBesar);

// Fungsi cepat tetap pakai 50.000 data
ukurWaktu(cariPasanganCepat, arrBesar, targetBesar);
```

```
// Diskusi

// Fungsi terbaik: cariPasanganCepat
// Karena:
// - Time complexity lebih baik (O(n))
// - Lebih cepat untuk data besar

// Trade-off:
// - Lambat: hemat memori (O(1)) tapi sangat lama
// - Cepat: lebih boros memori (O(n)) tapi jauh lebih efisien
```

5. Simpan file, lalu jalankan dari terminal

```
 latihan
Hasil Lambat: [ 1, 8 ]
Hasil Cepat : [ 4, 5 ]

=== Pengujian Data Besar ===
cariPasanganLambat => 4 ms | hasil: null
cariPasanganCepat => 3 ms | hasil: null
```

D. Bagian 4 — Mengenali Pola Kompleksitas

1. Buat file baru bernama 04-refactor.js di dalam folder praktikum-5/.

```
touch 04-refactor.js
```

2. Buka file 04-refactor.js di VS Code, lalu ketikkan kode berikut.

```
// 05-refactor.js — Refactoring kode berperforma buruk
// ═══════════════════════════════════════════════
// SKENARIO 1: Cek duplikat dalam array
// ═══════════════════════════════════════════════
// BURUK: O(n2) — nested loop
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j]) return true;
  return false;
}
```

```javascript
// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}
// ═══════════════════════════════════════════
// SKENARIO 2: Frekuensi kemunculan elemen
// ═══════════════════════════════════════════
// BURUK: O(n2) — indexOf di dalam loop
function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}
// BAIK: O(n) — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}
// ── Benchmark ──────────────────────────
const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);
let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n2):", Date.now() - t, "ms");
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");
const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array   : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]
```

3. Simpan file, lalu jalankan dari terminal.

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n2): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

Langkah 3 Simpan file, lalu jalankan dari terminal

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT  O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
```

E. TUGAS PRAKTIKUM

Langkah 1 Buat folder dan file tugas.

```
Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
$ mkdir tugas

Ars Com Lsm@DESKTOP-QGKPDN7 MINGW64 /d/2025573010058_PraktikumAlgoritmaDanStrukturData (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

TUGAS 1

Tugas 1: Analisis dan Refactor
1. Buat file tugas/tugas-1.js. 2. Terdapat tiga fungsi berikut yang perlu kamu analisis dan refactor: 3. • Fungsi A: menerima dua array, kembalikan array berisi elemen yang ada di keduanya (intersection). Implementasikan versi O(n2) dan O(n) menggunakan Set. 4. • Fungsi B: diberikan array string, kembalikan kelompok anagram. Contoh: ['eat','tea','tan','ate','nat','bat'] → [['eat','tea','ate'],['tan','nat'],['bat']]. Hint: sortir setiap kata sebagai key. 5. • Fungsi C: diberikan array angka positif, kembalikan true jika ada dua elemen yang jumlahnya sama dengan kuadrat elemen ketiga. Buat versi O(n3) dan O(n log n). 6. Untuk setiap fungsi: tulis Big O time & space dalam komentar, ukur waktu eksekusi keduanya dengan data besar. 7. Simpan dan jalankan: node tugas/tugas-1.js.

1. Buka file tugas-1.js dan ketikkan kode berikut

```
// ================================
// TUGAS 1 - Analisis & Refactor
// ================================

// ====================================
```

```
// FUNGSI A - INTERSECTION ARRAY
// ====================================

// Versi O(n^2)
function intersectionN2(arr1, arr2) {
  // Time: O(n^2), Space: O(1)
  const result = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }
  return result;
}

// Versi O(n) menggunakan Set
function intersectionN(arr1, arr2) {
  // Time: O(n), Space: O(n)
  const set2 = new Set(arr2);
  const result = [];

  for (let item of arr1) {
    if (set2.has(item)) {
      result.push(item);
    }
  }

  return [...new Set(result)]; // hapus duplikat
}

// ====================================
// FUNGSI B - KELOMPOK ANAGRAM
// ====================================

function groupAnagrams(words) {
  // Time: O(n log k), Space: O(n)
  // n = jumlah kata, k = panjang kata

  const map = new Map();

  for (let word of words) {
    const key = word.split("").sort().join("");
```

```
    if (!map.has(key)) {
      map.set(key, []);
    }

    map.get(key).push(word);
  }

  return Array.from(map.values());
}

// ====================================
// FUNGSI C - SUM = KUADRAT
// ====================================

// Versi O(n^3)
function sumSquareN3(arr) {
  // Time: O(n^3), Space: O(1)
  const n = arr.length;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      for (let k = 0; k < n; k++) {
        if (arr[i] + arr[j] === arr[k] * arr[k]) {
          return true;
        }
      }
    }
  }
  return false;
}

// Versi O(n log n)
function sumSquareOptimized(arr) {
  // Time: O(n log n), Space: O(1)
  arr.sort((a, b) => a - b);

  const n = arr.length;

  for (let k = n - 1; k >= 0; k--) {
    let left = 0;
    let right = k - 1;

    const target = arr[k] * arr[k];

    while (left <= right) {
```

```
      const sum = arr[left] + arr[right];

      if (sum === target) {
        return true;
      } else if (sum < target) {
        left++;
      } else {
        right--;
      }
    }
  }

  return false;
}

// ====================================
// TEST & PENGUKURAN WAKTU
// ====================================

// Data besar
const size = 5000;
const arr1 = Array.from({ length: size }, () =>
  Math.floor(Math.random() * 10000),
);
const arr2 = Array.from({ length: size }, () =>
  Math.floor(Math.random() * 10000),
);

const words = ["eat", "tea", "tan", "ate", "nat", "bat"];

// Fungsi A test
console.time("Intersection O(n^2)");
intersectionN2(arr1, arr2);
console.timeEnd("Intersection O(n^2)");

console.time("Intersection O(n)");
intersectionN(arr1, arr2);
console.timeEnd("Intersection O(n)");

// Fungsi B test
console.log("Anagram:", groupAnagrams(words));

// Fungsi C test
console.time("SumSquare O(n^3)");
sumSquareN3([1, 2, 3, 4, 5, 6, 7, 8, 9]);
```

```
console.timeEnd("SumSquare O(n^3)");

console.time("SumSquare O(n log n)");
sumSquareOptimized([1, 2, 3, 4, 5, 6, 7, 8, 9]);
console.timeEnd("SumSquare O(n log n)");
```

2. buka terminal dan liat hasilnya

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5/tugas (main)
$ node tugas-1
Intersection O(n^2): 44.041ms
Intersection O(n): 1.237ms
Anagram: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
SumSquare O(n^3): 0.052ms
SumSquare O(n log n): 0.074ms
```

TUGAS 2

1. Buat file tugas/tugas-2.js.

2. Buat function benchmarkSemua(ukuranData) yang menerima array ukuran data, lalu untuk setiap ukuran, mengukur dan mencetak waktu eksekusi dari empat fungsi:

3. • fn_O1(n) → hanya return n+1

4. • fn_On(n) → loop n kali, hitung jumlah

5. • fn_OnLogn(n) → simulasikan dengan loop luar n kali, loop dalam log2(n) kali

6. • fn_On2(n) → nested loop n×n

7. Panggil benchmarkSemua([100, 500, 1000, 5000, 10000]).

8. Amati dan tuliskan komentar di kode tentang pola pertumbuhan yang kamu observasi — apakah konsisten dengan teori Big O?

9. Simpan dan jalankan: node tugas/tugas-2.js.

1. Buka file tugas-1.js dan ketikkan kode berikut

```
// ========================================
// TUGAS 2 - Visualisasi Pertumbuhan Big O
// ========================================

// ==========================
// DEFINISI FUNGSI
// ==========================
```

```javascript
// O(1)
function fn_O1(n) {
  // Time: O(1), Space: O(1)
  return n + 1;
}

// O(n)
function fn_On(n) {
  // Time: O(n), Space: O(1)
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}

// O(n log n)
function fn_OnLogn(n) {
  // Time: O(n log n), Space: O(1)
  let count = 0;
  const logN = Math.floor(Math.log2(n));

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < logN; j++) {
      count++;
    }
  }
  return count;
}

// O(n^2)
function fn_On2(n) {
  // Time: O(n^2), Space: O(1)
  let count = 0;

  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }
  return count;
}

// ==========================
// BENCHMARK FUNCTION
```

```
// ==========================

function benchmarkSemua(ukuranData) {
  console.log("===== HASIL BENCHMARK =====");

  for (let n of ukuranData) {
    console.log(`\nUkuran data: ${n}`);

    console.time("O(1)");
    fn_O1(n);
    console.timeEnd("O(1)");

    console.time("O(n)");
    fn_On(n);
    console.timeEnd("O(n)");

    console.time("O(n log n)");
    fn_OnLogn(n);
    console.timeEnd("O(n log n)");

    console.time("O(n^2)");
    fn_On2(n);
    console.timeEnd("O(n^2)");
  }
}
// ==========================
// JALANKAN BENCHMARK
// ==========================

benchmarkSemua([100, 500, 1000, 5000, 10000]);

// ==========================
// ANALISIS HASIL (KOMENTAR)
// ==========================

/*
HASIL PENGAMATAN (umumnya):

1. O(1):
   - Waktu hampir sama untuk semua ukuran data
   - Tidak terpengaruh oleh besar kecilnya n

2. O(n):
   - Waktu bertambah secara linear
   - Jika n naik 2x, waktu kira-kira juga naik 2x
```

```
3. O(n log n):
   - Lebih cepat dari O(n^2), tapi lebih lambat dari O(n)
   - Pertumbuhan mulai terasa saat n besar

4. O(n^2):
   - Waktu meningkat sangat drastis
   - Jika n naik 10x, waktu bisa naik ~100x
   - Paling lambat di antara semua

KESIMPULAN:
- Pola pertumbuhan waktu eksekusi konsisten dengan teori Big O
- Semakin tinggi kompleksitas, semakin cepat waktu eksekusi meningkat
- O(1) paling efisien, O(n^2) paling tidak efisien untuk data besar
```

2. Buka dan liat hasilnya di terminal

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5/tugas (main)
$ node tugas-2
===== HASIL BENCHMARK =====

Ukuran data: 100
O(1): 0.024ms
O(n): 0.035ms
O(n log n): 0.064ms
O(n^2): 0.188ms

Ukuran data: 500
O(1): 0.013ms
O(n): 0.011ms
O(n log n): 0.067ms
O(n^2): 0.564ms

Ukuran data: 1000
O(1): 0.004ms
O(n): 0.03ms
O(n log n): 0.042ms
O(n^2): 1.153ms

Ukuran data: 5000
O(1): 0.003ms
O(n): 0.006ms
O(n log n): 0.08ms
O(n^2): 21.715ms

Ukuran data: 10000
O(1): 0.003ms
O(n): 0.012ms
O(n log n): 0.361ms
O(n^2): 82.461ms
```

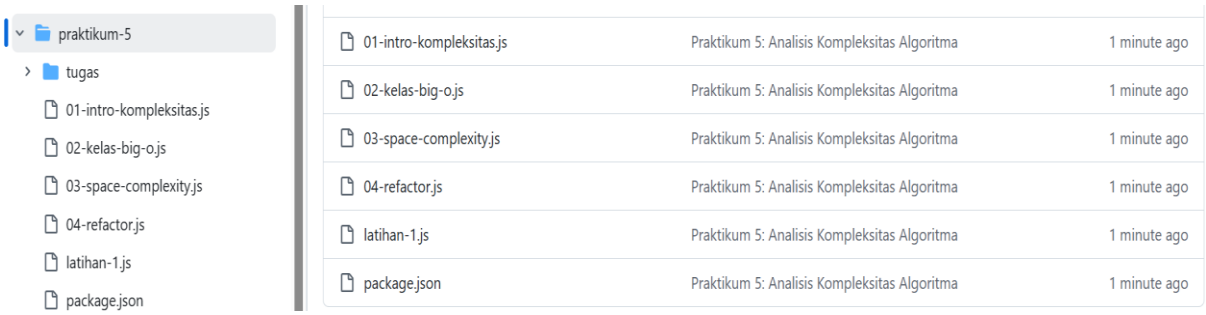
F. Langkah terakhir Submit ke GitHub

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat (main)
$ cd praktikum-5

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ git add .
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 72c6401] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 622 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/Latihan1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.82 KiB | 1.14 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/auliaillah-cell/2025573010070_praktikum_algoritma_dan_struktur_dat.git
   540cbb9..72c6401  main -> main
```



KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma merupakan hal yang sangat penting dalam pengembangan program. Big O Notation digunakan sebagai alat untuk mengukur seberapa efisien suatu algoritma dalam menangani data dengan ukuran yang berbeda. Melalui praktikum ini, dapat dipahami bahwa semakin kecil nilai kompleksitas suatu algoritma, maka semakin baik performanya, terutama ketika digunakan pada data dalam jumlah besar.

Hasil pengujian menunjukkan bahwa algoritma dengan kompleksitas O(1) memiliki waktu eksekusi yang sangat cepat dan tidak dipengaruhi oleh ukuran input. Sementara itu, algoritma dengan kompleksitas O(n) mengalami peningkatan waktu secara linear, dan algoritma dengan kompleksitas $O(n^2)$ mengalami peningkatan waktu yang sangat signifikan sehingga kurang efisien untuk data besar. Hal ini membuktikan bahwa pemilihan algoritma yang tepat sangat berpengaruh terhadap kinerja program.

Selain itu, praktikum ini juga menunjukkan adanya hubungan antara penggunaan waktu dan memori, di mana algoritma yang lebih cepat sering kali membutuhkan memori tambahan. Oleh karena itu, diperlukan pertimbangan yang tepat dalam memilih algoritma agar dapat mencapai keseimbangan antara efisiensi waktu dan penggunaan ruang.

Dengan demikian, pemahaman mengenai Big O Notation, Time Complexity, dan Space Complexity sangat penting bagi mahasiswa dalam menulis kode yang optimal, efisien, dan mampu menangani data dalam skala besar secara efektif.

```
ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat (main)
$ cd praktikum-5

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ git add .
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 72c6401] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 622 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/Latihan1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js

ASUS@LAPTOP-E52LLN2Q MINGW64 /d/2025573010070_praktikum_algoritma_dan_struktur_dat/praktikum-5 (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 6.82 KiB | 1.14 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/auliaillah-cell/2025573010070_praktikum_algoritma_dan_struktur_dat.git
   540cbb9..72c6401  main -> main
```